

PROJECT SPECIFICATIONS

ESG, Financial Performance & Time-Series Image (GAF) Dataset for Stock Risk Classification

GitHub: <https://github.com/ma24ahv/Week-11-Team-3>

1. Project

This project builds an application using multiple machine learning algorithms to classify S&P 500 companies into Low, Medium and High risk categories. It brings together ESG (Environmental, Social, Governance) risk indicators, company financial performance metrics and deep learning on time-series stock data encoded as Gramian Angular Field (GAF) images. The final output is an interactive Streamlit application that lets investors query the risk category of a company and see which features drove that prediction through SHAP explanations.

2. Research Question

The main research question is whether combining ESG scores, financial performance indicators, and angular field time series images can improve stock risk classification compared to models that only use one type of data. The project also looks at whether this combination can be a practical tool for investment decision-making.

Sub questions:

- Can angular image time series data of stock price history improve risk classification when integrated with a CNN-based classifier instead of just using tabular financial and ESG data alone?
- Which ESG and financial features are most likely to accurately predict the risk category of a company in the stock market?
- How do model predictions align with known high-risk events like sector downturns and ESG controversies that often cause fluctuations in the data?

3. Project Steps and Objectives

- First we collected the S&P 500 datasets for the project. We used four of them: ESG ratings, ESG metrics, financial data, and historical prices. We cleaned these datasets and then merged them into one feature matrix with all the information from the four sources. After merging and removing rows with missing values, we were left with 364 companies.
- We created risk labels for each company based on ESG risk score thresholds from Sustainalytics. Labels are low, medium, or high. We also used volatility measures to cross-check these labels.
- We generated GAF images from the 252-day (one year) normalized stock price series for all companies using the pyts Python package. Each image is 64x64 pixels.
- We compared four different models. The first was a CNN trained on the GAF images only. The second was a random forest trained on tabular ESG and financial features only. The third was a standalone ResNet50 model using transfer learning with

ImageNet weights. The fourth and final model was the hybrid ResNet50 + XGBoost, which combined image embeddings from ResNet50 with the 80 tabular features.

- We tuned the hybrid model using Optuna with 25 Bayesian optimization trials. We kept a record of every experiment so we could track what worked.
- We evaluated all models using weighted F1-score, AUC-ROC, accuracy, precision, recall, and confusion matrices. We also compared our results to what other research has found.
- We built the application using Streamlit. Users just enter a company ticker, and the app returns a risk classification plus a SHAP bar chart showing the top 15 features that influenced the prediction.
- We tested the app during a live demo. The app fetches price data from Yahoo Finance, generates the GAF image, builds the feature vector, and runs the prediction in real time.

4. Goals and Targets

- **Model Performance:** The hybrid model achieved a weighted F1-score of 0.9822 on the test set, which is well above the target of 0.80. This confirms the training was successful.
- **Multimodal Fusion:** The hybrid model combining image and tabular data scored significantly higher than any single modality model. The Random Forest (F1 = 0.8078) and the standalone ResNet50 (F1 = 0.4164) both performed worse on their own. This confirms the multimodal approach works better.
- **Transfer Learning:** We used ResNet50 pretrained on ImageNet instead of EfficientNet-B0. The final AUC-ROC on the test set was 0.9903, which is well above the 0.85 target. ResNet50 was chosen because its residual connections work better for the abstract patterns in GAF images.
- **Explainability:** SHAP TreeExplainer was used to generate feature importance for every prediction. The top features driving risk classification were environmentScore, ann_vol, beta, governanceScore, and price_sharpe. These were displayed as a bar chart in the app for every prediction.
- **App Deployment:** The Streamlit app runs on Google Colab via localtunnel. The app responds in under 3 seconds for most tickers. The interface is simple enough that anyone can use it without a technical background.
- **Dataset Coverage:** We ended up with 364 companies after the inner join merge across all four datasets. This is slightly below the original 400-company target due to missing ESG data for some tickers, but the dataset was still large enough to train and evaluate the models properly.

5. Key Performance Indicators (KPIs)

The following KPIs were set at the start to define project success. The table below shows the original targets and final results.

	KPI	Metric	Target	Final Result
1	Classification Accuracy	Weighted F1-score	≥ 0.80	0.9822

	KPI	Metric	Target	Final Result
2	Class Discrimination	AUC-ROC	≥ 0.85	0.9903
3	Minority Class Recall	Recall for High-Risk class	≥ 0.75	1.0000
4	Hybrid Model Gain	F1 delta vs best unimodal	$\geq +0.03$	+0.1744
5	Hyperparameter Coverage	Experiments logged	≥ 20 trials	25 trials
6	Data Completeness	Companies with full feature set	≥ 400 companies	364 companies
7	App Response Time	End-to-end inference latency	< 3 seconds	< 3 seconds
8	UAT Satisfaction	User rating (1–5 scale)	Mean ≥ 4.0	N/A
9	Explainability	SHAP plots generated	100% of tabular predictions	100%
10	Reproducibility	Results replicated from seed	100% reproducible pipeline	100% (seed=42)

Note on KPI 6: we ended up with 364 companies after merging all four datasets. Some S&P 500 tickers did not have complete ESG data available so they were dropped during the merge. The model still trained and evaluated well with 364 companies.

Note on KPI 8: UAT was not formally conducted at the end of the project. The app was tested during the live demo instead.

6. Models

- **CNN (Custom):** A custom convolutional neural network trained on the 64x64 GAF images. This was used as the image-only baseline. It reached around 45% validation accuracy on its own, which showed that image data alone without ESG and financial context was not enough.
- **ResNet50 (Transfer Learning):** We used ResNet50 pretrained on ImageNet weights instead of EfficientNet-B0. The model was frozen and used as a feature extractor. The global average pooling layer output gives a 2048-dimensional embedding per company. As a standalone image classifier, it achieved F1 = 0.4164 and AUC = 0.5352 which was poor on its own, but the embeddings were very useful in the hybrid model.
- **Hybrid ResNet50 + XGBoost:** This was the final model. The 2048-dim ResNet50 embeddings were concatenated with 80 tabular ESG and financial features to make a 2128-dimensional feature vector. This was passed to an XGBoost classifier tuned with 25 Optuna trials. Final results: F1 = 0.9822, AUC = 0.9903, accuracy = 98.18%, High-Risk Recall = 1.0000.

- Random Forest (Baseline): A Random Forest classifier trained on the 80 tabular features only. This was the tabular data baseline. It achieved F1 = 0.8078 and AUC = 0.9545 which was decent but still well below the hybrid model.

All models were compared on the same train/validation/test split (70/15/15, stratified by risk class). Hyperparameter tuning was done using Optuna (25 trials) for the hybrid model and a grid search approach for other models.

7. Team

Member	Primary Role	Secondary Role
Member 1 (Tamanna)	Team Leader	Model evaluation
Member 2 (Arafat)	Github Manager	Feature engineering
Member 3 (Eben)	Communication Leader	GAF image generation
Member 4 (Josh)	Scrum Master	Hyperparameter tuning
Member 5 (Mim)	Code Review Manager	UAT coordination
Member 6 (Sazzad)	Test Plan Manager	Literature review